JAVA TECH COMMUNITY

Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!

DAY 18 /100

JAVA INTERVIEW - CODING ROUND

TOP-10

NUMBER PROGRAMS

01 Write a program to check if a number is a prime number.

02 Write a program to find the factorial of a number.

03 Write a program to check if a number is an Armstrong number.

04 Write a program to reverse a number.

05 program to find the GCD of two numbers.

06 Write a program to check if a number is a perfect number.

07 Write a program to find the sum of digits of a number.

08 Write a program to convert a decimal number to binary.

09 Write a program to check if a year is a leap year.

10 Write a program to find the sum of the first N natural numbers.

01 Write a program to check if a number is a prime number.

```java
// Way 1: Without built-in methods
public class PrimeCheck {
    public static void main(String[] args) {
        int num = 29;
        boolean isPrime = true;

        if (num < 2) isPrime = false;
        for (int i = 2; i <= num / 2; i++) {
            if (num % i == 0) {
                isPrime = false;
                break;
            }
        }

        System.out.println(num + (isPrime ? " is Prime." : " is not Prime."));
    }
}
```

```java
// Way 2: Using square root for optimization
public class PrimeCheckOptimized {
    public static void main(String[] args) {
        int num = 29;
        boolean isPrime = true;

        if (num < 2) isPrime = false;
        for (int i = 2; i <= Math.sqrt(num); i++) {
            if (num % i == 0) {
                isPrime = false;
                break;
            }
        }

        System.out.println(num + (isPrime ? " is Prime." : " is not Prime."));
    }
}
```

02 Write a program to find the factorial of a number.

```java
// Way 1: Without built-in methods
public class Factorial {
    public static void main(String[] args) {
        int num = 5;
        int factorial = 1;

        for (int i = 1; i <= num; i++) {
            factorial *= i;
        }

        System.out.println("Factorial: " + factorial);
    }
}

// Way 2: Using recursion
public class FactorialRecursion {
    public static int factorial(int n) {
        return (n == 0 || n == 1) ? 1 : n * factorial(n - 1);
    }
    public static void main(String[] args) {
        int num = 5;
        System.out.println("Factorial: " + factorial(num));
    }
}
```

03 Write a program to check if a number is an Armstrong number.

```java
// Way 1: Without built-in methods
public class ArmstrongCheck {
    public static void main(String[] args) {
        int num = 153, sum = 0, temp = num;

        while (temp > 0) {
            int digit = temp % 10;
            sum += digit * digit * digit;
            temp /= 10;
        }

        System.out.println(num + (sum == num ? " is Armstrong." : " is not Armstrong."));
    }
}

// Way 2: Using a function
public class ArmstrongCheckFunction {
    public static boolean isArmstrong(int num) {
        int sum = 0, temp = num;
        while (temp > 0) {
            int digit = temp % 10;
            sum += digit * digit * digit;
            temp /= 10;
        }
        return sum == num;
    }
    public static void main(String[] args) {
        int num = 153;
        System.out.println(num + (isArmstrong(num) ? " is Armstrong." : " is not Armstrong."));
    }
}
```

04 Write a program to reverse a number.

```java
// Way 1: Without built-in methods
public class ReverseNumber {
    public static void main(String[] args) {
        int num = 1234, reversed = 0;

        while (num > 0) {
            reversed = reversed * 10 + num % 10;
            num /= 10;
        }

        System.out.println("Reversed Number: " + reversed);
    }
}

// Way 2: Using a function
public class ReverseNumberFunction {
    public static int reverse(int num) {
        int reversed = 0;
        while (num > 0) {
            reversed = reversed * 10 + num % 10;
            num /= 10;
        }
        return reversed;
    }
    public static void main(String[] args) {
        int num = 1234;
        System.out.println("Reversed Number: " + reverse(num));
    }
}
```

05 program to find the GCD of two numbers.

```java
// Way 1: Without built-in methods
public class GCD {
    public static void main(String[] args) {
        int a = 56, b = 98;
        while (a != b) {
            if (a > b) a -= b;
            else b -= a;
        }
        System.out.println("GCD: " + a);
    }
}

// Way 2: Using Euclidean algorithm (recursive)
public class GCDRecursive {
    public static int gcd(int a, int b) {
        return (b == 0) ? a : gcd(b, a % b);
    }
    public static void main(String[] args) {
        int a = 56, b = 98;
        System.out.println("GCD: " + gcd(a, b));
    }
}
```

06 Write a program to check if a number is a perfect number.

```java
// Way 1: Without built-in methods
public class PerfectNumber {
    public static void main(String[] args) {
        int num = 28, sum = 0;

        for (int i = 1; i <= num / 2; i++) {
            if (num % i == 0) sum += i;
        }

        System.out.println(num + (sum == num ? " is Perfect." : " is not Perfect."));
    }
}

// Way 2: Using a function
public class PerfectNumberFunction {
    public static boolean isPerfect(int num) {
        int sum = 0;
        for (int i = 1; i <= num / 2; i++) {
            if (num % i == 0) sum += i;
        }
        return sum == num;
    }
    public static void main(String[] args) {
        int num = 28;
        System.out.println(num + (isPerfect(num) ? " is Perfect." : " is not Perfect."));
    }
}
```

07 Write a program to find the sum of digits of a number.

```
// Way 1: Without built-in methods
public class SumOfDigits {
    public static void main(String[] args) {
        int num = 1234, sum = 0;

        while (num > 0) {
            sum += num % 10;
            num /= 10;
        }

        System.out.println("Sum of Digits: " + sum);
    }
}

// Way 2: Using a function
public class SumOfDigitsFunction {
    public static int sumOfDigits(int num) {
        int sum = 0;
        while (num > 0) {
            sum += num % 10;
            num /= 10;
        }
        return sum;
    }
    public static void main(String[] args) {
        int num = 1234;
        System.out.println("Sum of Digits: ' + sumOfDigits(num));
    }
}
```

08 Write a program to convert a decimal number to binary.

```
// Way 1: Without built-in methods
public class DecimalToBinary {
    public static void main(String[] args) {
        int num = 13; // Input number
        String binary = "";

        while (num > 0) {
            binary = (num % 2) + binary; // Append binary digits in reverse
            num /= 2;
        }

        System.out.println("Binary: " + binary);
    }
}

// Way 2: Using a function
public class DecimalToBinaryFunction {
    public static String toBinary(int num) {
        String binary = "";
        while (num > 0) {
            binary = (num % 2) + binary;
            num /= 2;
        }
        return binary;
    }
    public static void main(String[] args) {
        int num   13;
        System.out.println("Binary: " + toBinary(num));
    }
}
```

09 Write a program to check if a year is a leap year.

```
// Way 1: Without built-in methods
public class LeapYearCheck {
    public static void main(String[] args) {
        int year = 2024;

        if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
            System.out.println(year + " is a Leap Year.");
        } else {
            System.out.println(year + " is not a Leap Year.");
        }
    }
}

// Way 2: Using a reusable function
public class LeapYearFunction {
    public static boolean isLeapYear(int year) {
        return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    }
    public static void main(String[] args) {
        int year = 2024;
        System.out.println(year + (isLeapYear(year) ? " is a Leap Year." : " is not a Leap Year."));
    }
}
```

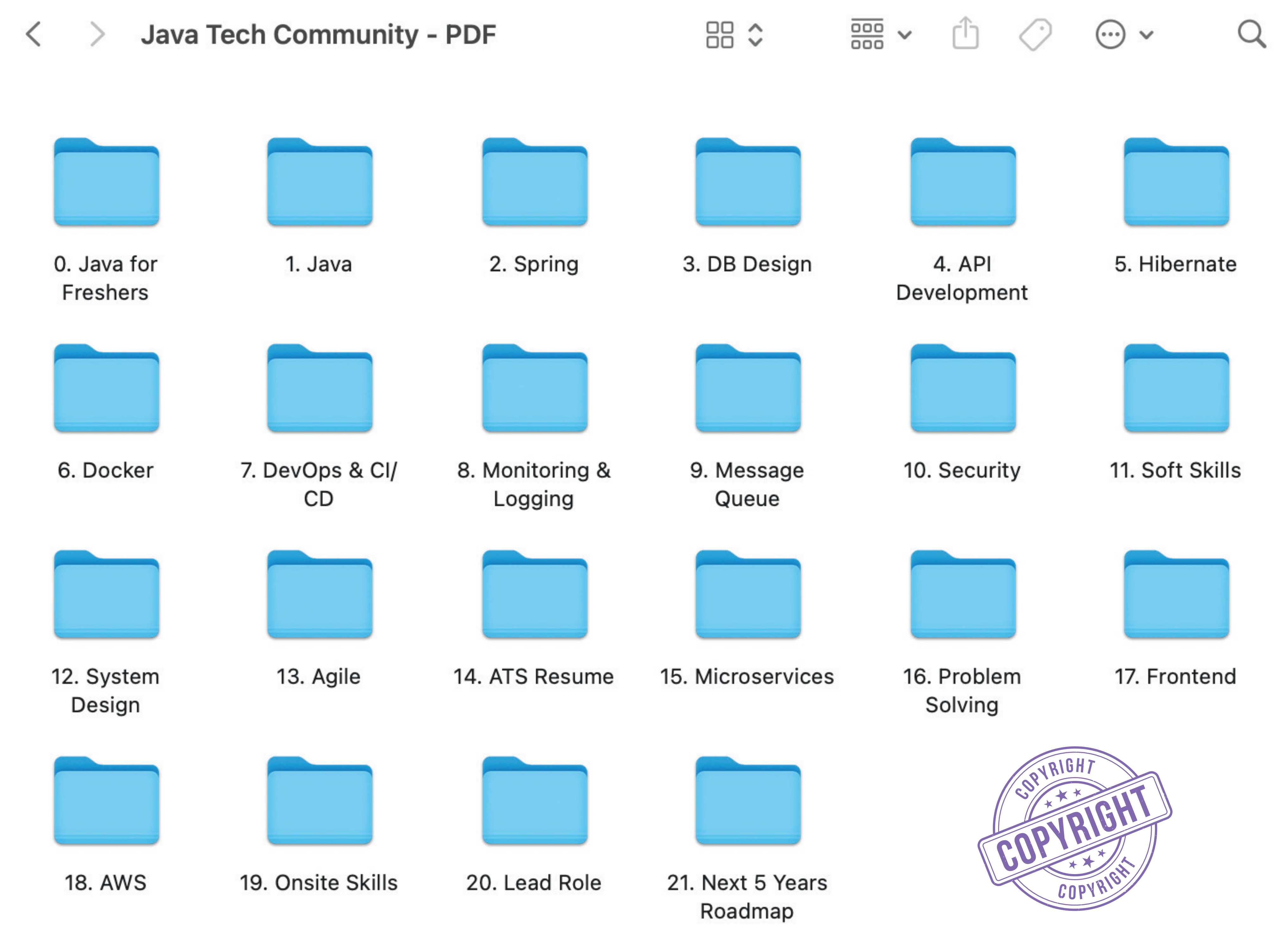
10 Write a program to find the sum of the first N natural numbers.

```
// Way 1: Without built-in methods
public class SumOfNaturalNumbers {
    public static void main(String[] args) {
        int n = 10; // Number up to which sum is needed
        int sum = 0;

        for (int i = 1; i <= n; i++) {
            sum += i;
        }

        System.out.println("Sum of first " + n + " natural numbers: " + sum);
    }
}

// Way 2: Using formula (n * (n + 1) / 2)
public class SumOfNaturalNumbersFormula {
    public static int sumOfNaturalNumbers(int n) {
        return n * (n + 1) / 2;
    }
    public static void main(String[] args) {
        int n = 10;
        System.out.println("Sum of first " + n + " natural numbers: " + sumOfNaturalNumbers(n));
    }
}
```

ALL THE RESOURCES YOU NEED TO CRACK ANY JAVA INTERVIEW – WHETHER YOU'RE A FRESHER, EXPERIENCED, OR AN EXPERT!

WE CAN'T WAIT TO SHARE THIS PDF WITH YOU! 🎉
OUR TEAM IS WORKING ON EACH SECTION TO MAKE IT COMPREHENSIVE AND USEFUL FOR YOU.

YOU'LL BE ABLE TO ACCESS IT IN JUST A FEW DAYS!
MAKE SURE TO FOLLOW OUR INSTAGRAM PAGE FOR UPDATES, AND WE'LL NOTIFY YOU ONCE IT'S AVAILABLE. STAY TUNED! 🚀



STAY UPDATED

- 📲 UPDATES WILL BE SHARED IN THIS COMMUNITY CHANNEL.

FOLLOW US ON INSTAGRAM

@JAVA_TECH_COMMUNITY FOR REAL-TIME UPDATES!